

```
In [18]: import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [ ]:
```

```
In [19]: df = pd.read_csv('/Users/kev/Desktop/fall 2025/data science/flights_prepared.csv')

# Keep only flights that departed
df = df[(df["CANCELLED"] == 0) & (df["DIVERTED"] == 0)]
df = df.dropna(subset=["DEP_DELAY"])
df["DEP_DELAY"] = df["DEP_DELAY"].clip(-10, 180)

# Convert date column
df["FL_DATE"] = pd.to_datetime(df["FL_DATE"])

# Sort for temporal features
df = df.sort_values(["OP_CARRIER_AIRLINE_ID", "TAIL_NUM", "FL_DATE", "DEP_HOUR"])

print(f"Dataset size: {len(df):,} flights")
```

Dataset size: 277,859 flights

/var/folders/zx/fx_7_y5911j0h_6w12_72m_00000gn/T/ipykernel_3289/4158139438.py:9: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.

```
df["FL_DATE"] = pd.to_datetime(df["FL_DATE"])
```

```
In [ ]:
```

```
In [20]: # --- Temporal Features ---
df["HOUR_SIN"] = np.sin(2 * np.pi * df["DEP_HOUR"] / 24)
df["HOUR_COS"] = np.cos(2 * np.pi * df["DEP_HOUR"] / 24)
```

```
df["MONTH_SIN"] = np.sin(2 * np.pi * df["MONTH"] / 12)
df["MONTH_COS"] = np.cos(2 * np.pi * df["MONTH"] / 12)
df["DAY_SIN"] = np.sin(2 * np.pi * df["DAY_OF_WEEK"] / 7)
df["DAY_COS"] = np.cos(2 * np.pi * df["DAY_OF_WEEK"] / 7)
```

In []:

```
In [21]: # --- Historical Aircraft Performance ---
# Last 3 flights for this tail number
for lag in [1, 2, 3]:
    df[f"TAIL_DELAY_LAG{lag}"] = df.groupby("TAIL_NUM")["DEP_DELAY"].shift(lag)

df["TAIL_AVG_DELAY_7D"] = df.groupby("TAIL_NUM")["DEP_DELAY"].transform(
    lambda x: x.rolling(window=7, min_periods=1).mean().shift(1)
)
```

In []:

```
In [22]: # --- Airline Performance ---
df["AIRLINE_HOUR_AVG_DELAY"] = df.groupby(["OP_CARRIER_AIRLINE_ID", "DEP_HOUR"])["DEP_DELAY"].transform(
    lambda x: x.expanding(min_periods=10).mean().shift(1)
)

df["AIRLINE_DAILY_AVG_DELAY"] = df.groupby(["OP_CARRIER_AIRLINE_ID", "FL_DATE"])["DEP_DELAY"].transform("m
```

In []:

```
In [23]: # --- Route Performance ---
df["ROUTE_ID"] = df["OP_CARRIER_AIRLINE_ID"].astype(str) + "_" + df["DEST_AIRPORT_ID"].astype(str)
df["ROUTE_AVG_DELAY"] = df.groupby("ROUTE_ID")["DEP_DELAY"].transform(
    lambda x: x.expanding(min_periods=20).mean().shift(1)
)
```

In []:

```
In [24]: # --- Airport Congestion (already in data but create more) ---
df["DEST_HOURLY_FLTS"] = df.groupby(["DEST_AIRPORT_ID", "FL_DATE", "DEP_HOUR"])["DEP_DELAY"].transform("co

# --- Time-based patterns ---
df["IS_MORNING_RUSH"] = ((df["DEP_HOUR"] >= 6) & (df["DEP_HOUR"] <= 9)).astype(int)
```

```

df["IS_EVENING_RUSH"] = ((df["DEP_HOUR"] >= 16) & (df["DEP_HOUR"] <= 19)).astype(int)
df["IS_WEEKEND"] = (df["DAY_OF_WEEK"].isin([6, 7])).astype(int)
df["IS_HOLIDAY_SEASON"] = (df["MONTH"].isin([11, 12, 6, 7])).astype(int)

# --- Weather interaction features ---
df["WEATHER_RUSH_HOUR"] = df["WEATHER_FLAG"] * (df["IS_MORNING_RUSH"] + df["IS_EVENING_RUSH"])

# Fill missing values in new features
lag_cols = [f"TAIL_DELAY_LAG{i}" for i in [1,2,3]] + ["TAIL_AVG_DELAY_7D", "AIRLINE_HOUR_AVG_DELAY", "ROUTE"]
df[lag_cols] = df[lag_cols].fillna(0)

print("✅ Feature engineering complete")

```

✅ Feature engineering complete

In []:

```

In [25]: # =====
# 3. FEATURE SELECTION
# =====

cat_int_features = ["OP_CARRIER_AIRLINE_ID", "DEST_AIRPORT_ID"]
cat_str_features = ["TAIL_NUM"]

numeric_features = [
    # Temporal (cyclical)
    "HOUR_SIN", "HOUR_COS",
    "MONTH_SIN", "MONTH_COS",
    "DAY_SIN", "DAY_COS",

    # Original temporal
    "YEAR", "DAY_OF_MONTH",

    # Weather
    "WEATHER_FLAG", "WEATHER_RUSH_HOUR",

    # Congestion
    "ORIGIN_HOURLY_FLTS", "DEST_HOURLY_FLTS",
    "ORIGIN_DAILY_AVG_DEP_DELAY",

    # Historical performance
    "TAIL_DELAY_LAG1", "TAIL_DELAY_LAG2", "TAIL_DELAY_LAG3",

```

```

    "TAIL_AVG_DELAY_7D",
    "AIRLINE_HOUR_AVG_DELAY", "AIRLINE_DAILY_AVG_DELAY",
    "ROUTE_AVG_DELAY",

    # Time patterns
    "IS_MORNING_RUSH", "IS_EVENING_RUSH",
    "IS_WEEKEND", "IS_HOLIDAY_SEASON"
]

# Target
y = df["DEP_DELAY"].astype("float32").values
X = df.copy()

# Fill missing values
X[numeric_features] = X[numeric_features].fillna(0)
X[cat_str_features] = X[cat_str_features].fillna("UNKNOWN")
X[cat_int_features] = X[cat_int_features].fillna(-1)

```

In []:

```

In [26]: # =====
# 4. TRAIN/VAL SPLIT (Time-based)
# =====

# Use last 20% as validation (time-based split)
split_idx = int(len(X) * 0.8)
X_train, X_val = X.iloc[:split_idx], X.iloc[split_idx:]
y_train, y_val = y[:split_idx], y[split_idx:]

print(f"Train: {len(X_train):,} | Val: {len(X_val):,}")

```

Train: 222,287 | Val: 55,572

In []:

```

In [27]: # =====
# 5. BUILD ENHANCED MODEL
# =====

def quantile_loss(q):
    def loss(y_true, y_pred):

```

```

        e = y_true - y_pred
        return tf.reduce_mean(tf.maximum(q * e, (q - 1) * e))
    return loss

quantiles = [0.5, 0.8, 0.95]

# --- Input layers ---
inputs = {}

for col in numeric_features:
    inputs[col] = keras.Input(shape=(1,), name=col, dtype=tf.float32)

for col in cat_int_features:
    inputs[col] = keras.Input(shape=(1,), name=col, dtype=tf.int32)

for col in cat_str_features:
    inputs[col] = keras.Input(shape=(1,), name=col, dtype=tf.string)

# --- Numeric encoding with normalization ---
numeric_data = np.array(X_train[numeric_features], dtype="float32")
normalizer = layers.Normalization()
normalizer.adapt(numeric_data)

def encode_numeric(inputs_dict):
    num_list = [inputs_dict[col] for col in numeric_features]
    x = layers.Concatenate()(num_list)
    return normalizer(x)

```

In []:

```

In [28]: # --- Integer categorical encoding ---
int_lookup = {}
int_embed = {}

for col in cat_int_features:
    vocab = np.unique(X_train[col].astype("int32"))
    lookup = layers.IntegerLookup()
    lookup.adapt(vocab)
    int_lookup[col] = lookup

    vocab_size = len(vocab) + 2

```

```
embed_dim = min(16, (vocab_size // 2) + 1)
int_embed[col] = layers.Embedding(vocab_size, embed_dim)

def encode_int(inputs_dict):
    outs = []
    for col in cat_int_features:
        idx = int_lookup[col](inputs_dict[col])
        emb = int_embed[col](idx)
        emb = layers.Reshape((emb.shape[-1],))(emb)
        outs.append(emb)
    return layers.Concatenate()(outs)

# --- String categorical encoding ---
str_lookup = {}
str_embed = {}

for col in cat_str_features:
    vocab = X_train[col].astype(str).unique()
    lookup = layers.StringLookup(max_tokens=2000) # Limit vocab size
    lookup.adapt(vocab)
    str_lookup[col] = lookup

    vocab_size = min(2000, len(vocab) + 2)
    embed_dim = 16
    str_embed[col] = layers.Embedding(vocab_size, embed_dim)

def encode_str(inputs_dict):
    outs = []
    for col in cat_str_features:
        idx = str_lookup[col](inputs_dict[col])
        emb = str_embed[col](idx)
        emb = layers.Reshape((emb.shape[-1],))(emb)
        outs.append(emb)
    return layers.Concatenate()(outs)

# --- Combine all features ---
num_enc = encode_numeric(inputs)
int_enc = encode_int(inputs)
str_enc = encode_str(inputs)

all_features = layers.Concatenate()([num_enc, int_enc, str_enc])
```

In []:

```
In [29]: # --- Enhanced architecture with residual connections ---
x = layers.Dense(256, activation="relu")(all_features)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.3)(x)

# Residual block 1
x1 = layers.Dense(256, activation="relu")(x)
x1 = layers.BatchNormalization()(x1)
x1 = layers.Dropout(0.3)(x1)
x = layers.Add()([x, x1])

x = layers.Dense(128, activation="relu")(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.3)(x)

# Residual block 2
x2 = layers.Dense(128, activation="relu")(x)
x2 = layers.BatchNormalization()(x2)
x2 = layers.Dropout(0.3)(x2)
x = layers.Add()([x, x2])

x = layers.Dense(64, activation="relu")(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.2)(x)

x = layers.Dense(32, activation="relu")(x)
```

In []:

```
In [30]: # --- Output heads for each quantile ---
outputs = []
for q in quantiles:
    out = layers.Dense(1, name=f"q{q}")(x)
    outputs.append(out)

# --- Build model ---
model = keras.Model(inputs=inputs, outputs=outputs)
```

```
losses = {f"q{q}": quantile_loss(q) for q in quantiles}

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=5e-4),
    loss=losses
)

print("\n" + "="*50)
print("MODEL ARCHITECTURE")
print("="*50)
model.summary()
```

```
=====
MODEL ARCHITECTURE
=====
Model: "functional"
```

Layer (type)	Output Shape	Param #	Connected to
OP_CARRIER_AIRLINE... (InputLayer)	(None, 1)	0	—
DEST_AIRPORT_ID (InputLayer)	(None, 1)	0	—
TAIL_NUM (InputLayer)	(None, 1)	0	—
integer_lookup (IntegerLookup)	(None, 1)	0	OP_CARRIER_AIRLI...
integer_lookup_1 (IntegerLookup)	(None, 1)	0	DEST_AIRPORT_ID [...
string_lookup (StringLookup)	(None, 1)	0	TAIL_NUM[0][0]
HOUR_SIN (InputLayer)	(None, 1)	0	—
HOUR_COS (InputLayer)	(None, 1)	0	—
MONTH_SIN (InputLayer)	(None, 1)	0	—
MONTH_COS (InputLayer)	(None, 1)	0	—
DAY_SIN (InputLayer)	(None, 1)	0	—
DAY_COS (InputLayer)	(None, 1)	0	—
YEAR (InputLayer)	(None, 1)	0	—
DAY_OF_MONTH (InputLayer)	(None, 1)	0	—
WEATHER_FLAG	(None, 1)	0	—

(InputLayer)			
WEATHER_RUSH_HOUR (InputLayer)	(None, 1)	0	–
ORIGIN_HOURLY_FLTS (InputLayer)	(None, 1)	0	–
DEST_HOURLY_FLTS (InputLayer)	(None, 1)	0	–
ORIGIN_DAILY_AVG_D... (InputLayer)	(None, 1)	0	–
TAIL_DELAY_LAG1 (InputLayer)	(None, 1)	0	–
TAIL_DELAY_LAG2 (InputLayer)	(None, 1)	0	–
TAIL_DELAY_LAG3 (InputLayer)	(None, 1)	0	–
TAIL_AVG_DELAY_7D (InputLayer)	(None, 1)	0	–
AIRLINE_HOUR_AVG_D... (InputLayer)	(None, 1)	0	–
AIRLINE_DAILY_AVG_... (InputLayer)	(None, 1)	0	–
ROUTE_AVG_DELAY (InputLayer)	(None, 1)	0	–
IS_MORNING_RUSH (InputLayer)	(None, 1)	0	–
IS_EVENING_RUSH (InputLayer)	(None, 1)	0	–
IS_WEEKEND (InputLayer)	(None, 1)	0	–
IS_HOLIDAY_SEASON	(None, 1)	0	–

(InputLayer)			
embedding (Embedding)	(None, 1, 5)	45	integer_lookup[0...
embedding_1 (Embedding)	(None, 1, 16)	1,248	integer_lookup_1...
embedding_2 (Embedding)	(None, 1, 16)	32,000	string_lookup[0]...
concatenate (Concatenate)	(None, 24)	0	HOUR_SIN[0][0], HOUR_COS[0][0], MONTH_SIN[0][0], MONTH_COS[0][0], DAY_SIN[0][0], DAY_COS[0][0], YEAR[0][0], DAY_OF_MONTH[0][... WEATHER_FLAG[0][... WEATHER_RUSH_HOU... ORIGIN_HOURLY_FL... DEST_HOURLY_FLTS... ORIGIN_DAILY_AVG... TAIL_DELAY_LAG1[... TAIL_DELAY_LAG2[... TAIL_DELAY_LAG3[... TAIL_AVG_DELAY_7... AIRLINE_HOUR_AVG... AIRLINE_DAILY_AV... ROUTE_AVG_DELAY[... IS_MORNING_RUSH[... IS_EVENING_RUSH[... IS_WEEKEND[0][0], IS_HOLIDAY_SEASO...
reshape (Reshape)	(None, 5)	0	embedding[0][0]
reshape_1 (Reshape)	(None, 16)	0	embedding_1[0][0]
reshape_2 (Reshape)	(None, 16)	0	embedding_2[0][0]
normalization (Normalization)	(None, 24)	49	concatenate[0][0]

concatenate_1 (Concatenate)	(None, 21)	0	reshape[0][0], reshape_1[0][0]
concatenate_2 (Concatenate)	(None, 16)	0	reshape_2[0][0]
concatenate_3 (Concatenate)	(None, 61)	0	normalization[0]... concatenate_1[0]... concatenate_2[0]...
dense (Dense)	(None, 256)	15,872	concatenate_3[0]...
batch_normalization (BatchNormalizatio...	(None, 256)	1,024	dense[0][0]
dropout (Dropout)	(None, 256)	0	batch_normalizat...
dense_1 (Dense)	(None, 256)	65,792	dropout[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 256)	1,024	dense_1[0][0]
dropout_1 (Dropout)	(None, 256)	0	batch_normalizat...
add (Add)	(None, 256)	0	dropout[0][0], dropout_1[0][0]
dense_2 (Dense)	(None, 128)	32,896	add[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 128)	512	dense_2[0][0]
dropout_2 (Dropout)	(None, 128)	0	batch_normalizat...
dense_3 (Dense)	(None, 128)	16,512	dropout_2[0][0]
batch_normalizatio... (BatchNormalizatio...	(None, 128)	512	dense_3[0][0]
dropout_3 (Dropout)	(None, 128)	0	batch_normalizat...
add_1 (Add)	(None, 128)	0	dropout_2[0][0], dropout_3[0][0]
dense_4 (Dense)	(None, 64)	8,256	add_1[0][0]

batch_normalizatio... (BatchNormalizatio...	(None, 64)	256	dense_4[0][0]
dropout_4 (Dropout)	(None, 64)	0	batch_normalizat...
dense_5 (Dense)	(None, 32)	2,080	dropout_4[0][0]
q0.5 (Dense)	(None, 1)	33	dense_5[0][0]
q0.8 (Dense)	(None, 1)	33	dense_5[0][0]
q0.95 (Dense)	(None, 1)	33	dense_5[0][0]

Total params: 178,177 (696.01 KB)

Trainable params: 176,464 (689.31 KB)

Non-trainable params: 1,713 (6.70 KB)

In []:

In [31]:

```
# =====
# 6. CREATE DATASETS
# =====

def df_to_ds(df, y, shuffle=True, batch_size=2048):
    feats = {}
    for col in numeric_features + cat_int_features + cat_str_features:
        feats[col] = df[col].values
    labels = {f"q{q}": y for q in quantiles}
    ds = tf.data.Dataset.from_tensor_slices((feats, labels))
    if shuffle:
        ds = ds.shuffle(10000)
    return ds.batch(batch_size).prefetch(tf.data.AUTOTUNE)

train_ds = df_to_ds(X_train, y_train, shuffle=True)
val_ds = df_to_ds(X_val, y_val, shuffle=False)
```

In []:

```
In [32]: # =====  
# 7. TRAINING WITH CALLBACKS  
# =====  
  
early_stop = keras.callbacks.EarlyStopping(  
    monitor='val_loss',  
    patience=5,  
    restore_best_weights=True,  
    verbose=1  
)  
  
reduce_lr = keras.callbacks.ReduceLRonPlateau(  
    monitor='val_loss',  
    factor=0.5,  
    patience=3,  
    min_lr=1e-6,  
    verbose=1  
)  
  
print("\n" + "="*50)  
print("TRAINING")  
print("="*50)  
  
history = model.fit(  
    train_ds,  
    validation_data=val_ds,  
    epochs=50,  
    callbacks=[early_stop, reduce_lr],  
    verbose=1  
)
```

=====

TRAINING

=====

Epoch 1/50

109/109  **32s** 273ms/step - loss: 35.1267 - q0.5_loss: 9.2679 - q0.8_loss: 12.5193 - q0.95_loss: 13.3558 - val_loss: 27.9755 - val_q0.5_loss: 7.8366 - val_q0.8_loss: 10.0958 - val_q0.95_loss: 10.3659 - learning_rate: 5.0000e-04

Epoch 2/50

109/109  **29s** 270ms/step - loss: 28.0372 - q0.5_loss: 8.8258 - q0.8_loss: 10.2625 - q0.95_loss: 8.9836 - val_loss: 23.5432 - val_q0.5_loss: 7.5547 - val_q0.8_loss: 8.7323 - val_q0.95_loss: 7.4223 - learning_rate: 5.0000e-04

Epoch 3/50

109/109  **44s** 409ms/step - loss: 24.5315 - q0.5_loss: 8.6417 - q0.8_loss: 9.2561 - q0.95_loss: 6.6521 - val_loss: 21.3847 - val_q0.5_loss: 7.4996 - val_q0.8_loss: 8.1387 - val_q0.95_loss: 5.8612 - learning_rate: 5.0000e-04

Epoch 4/50

109/109  **35s** 323ms/step - loss: 23.3468 - q0.5_loss: 8.5396 - q0.8_loss: 9.0903 - q0.95_loss: 5.7328 - val_loss: 20.5168 - val_q0.5_loss: 7.4880 - val_q0.8_loss: 7.9673 - val_q0.95_loss: 5.1698 - learning_rate: 5.0000e-04

Epoch 5/50

109/109  **21s** 194ms/step - loss: 22.8661 - q0.5_loss: 8.4829 - q0.8_loss: 9.0466 - q0.95_loss: 5.3491 - val_loss: 20.0543 - val_q0.5_loss: 7.4353 - val_q0.8_loss: 7.8921 - val_q0.95_loss: 4.8211 - learning_rate: 5.0000e-04

Epoch 6/50

109/109  **36s** 333ms/step - loss: 22.5760 - q0.5_loss: 8.4395 - q0.8_loss: 9.0102 - q0.95_loss: 5.1511 - val_loss: 19.8153 - val_q0.5_loss: 7.3885 - val_q0.8_loss: 7.8511 - val_q0.95_loss: 4.6726 - learning_rate: 5.0000e-04

Epoch 7/50

109/109  **15s** 143ms/step - loss: 22.3868 - q0.5_loss: 8.4138 - q0.8_loss: 8.9636 - q0.95_loss: 5.0291 - val_loss: 19.6806 - val_q0.5_loss: 7.3501 - val_q0.8_loss: 7.8282 - val_q0.95_loss: 4.6004 - learning_rate: 5.0000e-04

Epoch 8/50

109/109  **32s** 294ms/step - loss: 22.2451 - q0.5_loss: 8.3658 - q0.8_loss: 8.9385 - q0.95_loss: 4.9632 - val_loss: 19.6082 - val_q0.5_loss: 7.3562 - val_q0.8_loss: 7.8184 - val_q0.95_loss: 4.5340 - learning_rate: 5.0000e-04

Epoch 9/50

109/109  **18s** 169ms/step - loss: 22.1201 - q0.5_loss: 8.3386 - q0.8_loss: 8.9099 - q0.95_loss: 4.9021 - val_loss: 19.5078 - val_q0.5_loss: 7.3221 - val_q0.8_loss: 7.8058 - val_q0.95_loss: 4.4858 - learning_rate: 5.0000e-04

Epoch 10/50

109/109  **10s** 88ms/step - loss: 22.0522 - q0.5_loss: 8.3254 - q0.8_loss: 8.8862 - q0.95_loss: 4.8670 - val_loss: 19.4366 - val_q0.5_loss: 7.3101 - val_q0.8_loss: 7.7900 - val_q0.95_loss: 4.4386 -

```
learning_rate: 5.0000e-04
Epoch 11/50
109/109  25s 231ms/step - loss: 21.9272 - q0.5_loss: 8.3002 - q0.8_loss: 8.8252 - q0.95_loss: 4.8206 - val_loss: 19.3545 - val_q0.5_loss: 7.2993 - val_q0.8_loss: 7.7588 - val_q0.95_loss: 4.3975 - learning_rate: 5.0000e-04
Epoch 12/50
109/109  4s 37ms/step - loss: 21.8049 - q0.5_loss: 8.2614 - q0.8_loss: 8.7796 - q0.95_loss: 4.7792 - val_loss: 19.2902 - val_q0.5_loss: 7.2458 - val_q0.8_loss: 7.7464 - val_q0.95_loss: 4.3977 - learning_rate: 5.0000e-04
Epoch 13/50
109/109  9s 79ms/step - loss: 21.7394 - q0.5_loss: 8.2404 - q0.8_loss: 8.7638 - q0.95_loss: 4.7524 - val_loss: 19.2357 - val_q0.5_loss: 7.2209 - val_q0.8_loss: 7.7338 - val_q0.95_loss: 4.3865 - learning_rate: 5.0000e-04
Epoch 14/50
109/109  7s 63ms/step - loss: 21.6249 - q0.5_loss: 8.2057 - q0.8_loss: 8.7103 - q0.95_loss: 4.7226 - val_loss: 19.1702 - val_q0.5_loss: 7.2120 - val_q0.8_loss: 7.7124 - val_q0.95_loss: 4.3447 - learning_rate: 5.0000e-04
Epoch 15/50
109/109  7s 61ms/step - loss: 21.5778 - q0.5_loss: 8.1928 - q0.8_loss: 8.6846 - q0.95_loss: 4.7118 - val_loss: 19.2118 - val_q0.5_loss: 7.2068 - val_q0.8_loss: 7.7335 - val_q0.95_loss: 4.3771 - learning_rate: 5.0000e-04
Epoch 16/50
109/109  4s 38ms/step - loss: 21.4954 - q0.5_loss: 8.1789 - q0.8_loss: 8.6554 - q0.95_loss: 4.6771 - val_loss: 19.1770 - val_q0.5_loss: 7.1778 - val_q0.8_loss: 7.7166 - val_q0.95_loss: 4.3895 - learning_rate: 5.0000e-04
Epoch 17/50
108/109  0s 59ms/step - loss: 19.8423 - q0.5_loss: 7.3449 - q0.8_loss: 7.9694 - q0.95_loss: 4.5280
Epoch 17: ReduceLRonPlateau reducing learning rate to 0.0002500000118743628.
109/109  7s 61ms/step - loss: 21.4347 - q0.5_loss: 8.1753 - q0.8_loss: 8.6301 - q0.95_loss: 4.6594 - val_loss: 19.1885 - val_q0.5_loss: 7.1754 - val_q0.8_loss: 7.7357 - val_q0.95_loss: 4.3856 - learning_rate: 5.0000e-04
Epoch 18/50
109/109  6s 59ms/step - loss: 21.4625 - q0.5_loss: 8.2065 - q0.8_loss: 8.6219 - q0.95_loss: 4.6478 - val_loss: 19.0488 - val_q0.5_loss: 7.0321 - val_q0.8_loss: 7.7136 - val_q0.95_loss: 4.4156 - learning_rate: 2.5000e-04
Epoch 19/50
109/109  4s 39ms/step - loss: 21.3148 - q0.5_loss: 8.1491 - q0.8_loss: 8.5732 - q0.95_loss: 4.6121 - val_loss: 19.0106 - val_q0.5_loss: 7.0335 - val_q0.8_loss: 7.6982 - val_q0.95_loss: 4.3947 - learning_rate: 2.5000e-04
Epoch 20/50
109/109  5s 45ms/step - loss: 21.2741 - q0.5_loss: 8.1271 - q0.8_loss: 8.5567 - q0.95_loss: 4.5888
```

oss: 4.6109 - val_loss: 19.0154 - val_q0.5_loss: 7.0265 - val_q0.8_loss: 7.6978 - val_q0.95_loss: 4.4102 -
learning_rate: 2.5000e-04
Epoch 21/50
109/109  5s 48ms/step - loss: 21.1992 - q0.5_loss: 8.1093 - q0.8_loss: 8.5236 - q0.95_l
oss: 4.5867 - val_loss: 18.9945 - val_q0.5_loss: 7.0147 - val_q0.8_loss: 7.6982 - val_q0.95_loss: 4.4021 -
learning_rate: 2.5000e-04
Epoch 22/50
109/109  4s 35ms/step - loss: 21.1538 - q0.5_loss: 8.0929 - q0.8_loss: 8.5048 - q0.95_l
oss: 4.5708 - val_loss: 18.9830 - val_q0.5_loss: 7.0185 - val_q0.8_loss: 7.6876 - val_q0.95_loss: 4.3970 -
learning_rate: 2.5000e-04
Epoch 23/50
109/109  7s 67ms/step - loss: 21.1048 - q0.5_loss: 8.0824 - q0.8_loss: 8.4807 - q0.95_l
oss: 4.5703 - val_loss: 18.9981 - val_q0.5_loss: 7.0076 - val_q0.8_loss: 7.7013 - val_q0.95_loss: 4.4098 -
learning_rate: 2.5000e-04
Epoch 24/50
109/109  7s 66ms/step - loss: 21.0600 - q0.5_loss: 8.0719 - q0.8_loss: 8.4713 - q0.95_l
oss: 4.5481 - val_loss: 18.9937 - val_q0.5_loss: 6.9996 - val_q0.8_loss: 7.7018 - val_q0.95_loss: 4.4182 -
learning_rate: 2.5000e-04
Epoch 25/50
109/109  4s 33ms/step - loss: 20.9850 - q0.5_loss: 8.0515 - q0.8_loss: 8.4317 - q0.95_l
oss: 4.5236 - val_loss: 18.9821 - val_q0.5_loss: 6.9971 - val_q0.8_loss: 7.6944 - val_q0.95_loss: 4.4155 -
learning_rate: 2.5000e-04
Epoch 26/50
109/109  7s 67ms/step - loss: 20.9654 - q0.5_loss: 8.0503 - q0.8_loss: 8.4203 - q0.95_l
oss: 4.5082 - val_loss: 18.9654 - val_q0.5_loss: 6.9998 - val_q0.8_loss: 7.6869 - val_q0.95_loss: 4.4052 -
learning_rate: 2.5000e-04
Epoch 27/50
109/109  5s 47ms/step - loss: 20.9317 - q0.5_loss: 8.0446 - q0.8_loss: 8.4137 - q0.95_l
oss: 4.5012 - val_loss: 18.9672 - val_q0.5_loss: 7.0002 - val_q0.8_loss: 7.6916 - val_q0.95_loss: 4.4009 -
learning_rate: 2.5000e-04
Epoch 28/50
109/109  7s 62ms/step - loss: 20.8372 - q0.5_loss: 8.0121 - q0.8_loss: 8.3627 - q0.95_l
oss: 4.4761 - val_loss: 18.9747 - val_q0.5_loss: 6.9964 - val_q0.8_loss: 7.6902 - val_q0.95_loss: 4.4171 -
learning_rate: 2.5000e-04
Epoch 29/50
109/109  4s 40ms/step - loss: 20.8004 - q0.5_loss: 7.9946 - q0.8_loss: 8.3535 - q0.95_l
oss: 4.4650 - val_loss: 18.9572 - val_q0.5_loss: 6.9882 - val_q0.8_loss: 7.6919 - val_q0.95_loss: 4.4042 -
learning_rate: 2.5000e-04
Epoch 30/50
109/109  6s 56ms/step - loss: 20.7499 - q0.5_loss: 7.9888 - q0.8_loss: 8.3276 - q0.95_l
oss: 4.4490 - val_loss: 18.9497 - val_q0.5_loss: 6.9931 - val_q0.8_loss: 7.6802 - val_q0.95_loss: 4.4040 -
learning_rate: 2.5000e-04

Epoch 31/50

109/109  **5s** 45ms/step - loss: 20.7161 - q0.5_loss: 7.9779 - q0.8_loss: 8.3139 - q0.95_loss: 4.4406 - val_loss: 18.9235 - val_q0.5_loss: 6.9829 - val_q0.8_loss: 7.6695 - val_q0.95_loss: 4.3999 - learning_rate: 2.5000e-04

Epoch 32/50

109/109  **5s** 42ms/step - loss: 20.6618 - q0.5_loss: 7.9682 - q0.8_loss: 8.2838 - q0.95_loss: 4.4310 - val_loss: 18.9353 - val_q0.5_loss: 6.9753 - val_q0.8_loss: 7.6803 - val_q0.95_loss: 4.4116 - learning_rate: 2.5000e-04

Epoch 33/50

109/109  **5s** 47ms/step - loss: 20.6295 - q0.5_loss: 7.9561 - q0.8_loss: 8.2744 - q0.95_loss: 4.4175 - val_loss: 18.9351 - val_q0.5_loss: 6.9838 - val_q0.8_loss: 7.6773 - val_q0.95_loss: 4.4062 - learning_rate: 2.5000e-04

Epoch 34/50

106/109  **0s** 59ms/step - loss: 18.8888 - q0.5_loss: 7.0551 - q0.8_loss: 7.5509 - q0.95_loss: 4.2829

Epoch 34: ReduceLROnPlateau reducing learning rate to 0.0001250000059371814.

109/109  **7s** 60ms/step - loss: 20.5796 - q0.5_loss: 7.9433 - q0.8_loss: 8.2422 - q0.95_loss: 4.4099 - val_loss: 18.9401 - val_q0.5_loss: 6.9885 - val_q0.8_loss: 7.6835 - val_q0.95_loss: 4.4064 - learning_rate: 2.5000e-04

Epoch 35/50

109/109  **6s** 56ms/step - loss: 20.5680 - q0.5_loss: 7.9411 - q0.8_loss: 8.2451 - q0.95_loss: 4.4028 - val_loss: 18.9331 - val_q0.5_loss: 7.0096 - val_q0.8_loss: 7.6649 - val_q0.95_loss: 4.3988 - learning_rate: 1.2500e-04

Epoch 36/50

109/109  **24s** 220ms/step - loss: 20.4888 - q0.5_loss: 7.9137 - q0.8_loss: 8.2030 - q0.95_loss: 4.3837 - val_loss: 18.9223 - val_q0.5_loss: 7.0049 - val_q0.8_loss: 7.6658 - val_q0.95_loss: 4.3916 - learning_rate: 1.2500e-04

Epoch 37/50

109/109  **14s** 128ms/step - loss: 20.4732 - q0.5_loss: 7.9115 - q0.8_loss: 8.1996 - q0.95_loss: 4.3788 - val_loss: 18.9072 - val_q0.5_loss: 7.0033 - val_q0.8_loss: 7.6587 - val_q0.95_loss: 4.3854 - learning_rate: 1.2500e-04

Epoch 38/50

109/109  **12s** 108ms/step - loss: 20.4422 - q0.5_loss: 7.9108 - q0.8_loss: 8.1979 - q0.95_loss: 4.3572 - val_loss: 18.8882 - val_q0.5_loss: 6.9969 - val_q0.8_loss: 7.6554 - val_q0.95_loss: 4.3788 - learning_rate: 1.2500e-04

Epoch 39/50

109/109  **12s** 114ms/step - loss: 20.3975 - q0.5_loss: 7.8951 - q0.8_loss: 8.1701 - q0.95_loss: 4.3508 - val_loss: 18.8944 - val_q0.5_loss: 6.9982 - val_q0.8_loss: 7.6550 - val_q0.95_loss: 4.3841 - learning_rate: 1.2500e-04

Epoch 40/50

109/109  **16s** 143ms/step - loss: 20.4059 - q0.5_loss: 7.8993 - q0.8_loss: 8.1746 - q0.95_loss: 4.3578 - val_loss: 18.8820 - val_q0.5_loss: 6.9931 - val_q0.8_loss: 7.6517 - val_q0.95_loss: 4.3798

```
- learning_rate: 1.2500e-04
Epoch 41/50
109/109 ————— 7s 68ms/step - loss: 20.4011 - q0.5_loss: 7.9022 - q0.8_loss: 8.1672 - q0.95_l
oss: 4.3554 - val_loss: 18.8847 - val_q0.5_loss: 6.9954 - val_q0.8_loss: 7.6489 - val_q0.95_loss: 4.3840 -
learning_rate: 1.2500e-04
Epoch 42/50
109/109 ————— 9s 83ms/step - loss: 20.3513 - q0.5_loss: 7.8802 - q0.8_loss: 8.1407 - q0.95_l
oss: 4.3511 - val_loss: 18.8920 - val_q0.5_loss: 7.0054 - val_q0.8_loss: 7.6478 - val_q0.95_loss: 4.3858 -
learning_rate: 1.2500e-04
Epoch 43/50
109/109 ————— 10s 93ms/step - loss: 20.3399 - q0.5_loss: 7.8760 - q0.8_loss: 8.1436 - q0.95_
loss: 4.3464 - val_loss: 18.8680 - val_q0.5_loss: 7.0041 - val_q0.8_loss: 7.6342 - val_q0.95_loss: 4.3774 -
learning_rate: 1.2500e-04
Epoch 44/50
109/109 ————— 11s 97ms/step - loss: 20.3194 - q0.5_loss: 7.8743 - q0.8_loss: 8.1224 - q0.95_
loss: 4.3428 - val_loss: 18.8618 - val_q0.5_loss: 7.0044 - val_q0.8_loss: 7.6337 - val_q0.95_loss: 4.3675 -
learning_rate: 1.2500e-04
Epoch 45/50
109/109 ————— 10s 87ms/step - loss: 20.2727 - q0.5_loss: 7.8598 - q0.8_loss: 8.1081 - q0.95_
loss: 4.3235 - val_loss: 18.8609 - val_q0.5_loss: 6.9983 - val_q0.8_loss: 7.6362 - val_q0.95_loss: 4.3725 -
learning_rate: 1.2500e-04
Epoch 46/50
109/109 ————— 12s 111ms/step - loss: 20.2781 - q0.5_loss: 7.8568 - q0.8_loss: 8.1144 - q0.95
_loss: 4.3311 - val_loss: 18.8391 - val_q0.5_loss: 6.9956 - val_q0.8_loss: 7.6269 - val_q0.95_loss: 4.3651
- learning_rate: 1.2500e-04
Epoch 47/50
109/109 ————— 10s 94ms/step - loss: 20.2382 - q0.5_loss: 7.8513 - q0.8_loss: 8.0967 - q0.95_
loss: 4.3133 - val_loss: 18.8298 - val_q0.5_loss: 6.9888 - val_q0.8_loss: 7.6233 - val_q0.95_loss: 4.3622 -
learning_rate: 1.2500e-04
Epoch 48/50
109/109 ————— 15s 142ms/step - loss: 20.2061 - q0.5_loss: 7.8485 - q0.8_loss: 8.0788 - q0.95
_loss: 4.2969 - val_loss: 18.8272 - val_q0.5_loss: 6.9936 - val_q0.8_loss: 7.6195 - val_q0.95_loss: 4.3594
- learning_rate: 1.2500e-04
Epoch 49/50
109/109 ————— 22s 205ms/step - loss: 20.2142 - q0.5_loss: 7.8468 - q0.8_loss: 8.0794 - q0.95
_loss: 4.3082 - val_loss: 18.8119 - val_q0.5_loss: 6.9919 - val_q0.8_loss: 7.6142 - val_q0.95_loss: 4.3545
- learning_rate: 1.2500e-04
Epoch 50/50
109/109 ————— 18s 163ms/step - loss: 20.1920 - q0.5_loss: 7.8412 - q0.8_loss: 8.0757 - q0.95
_loss: 4.3017 - val_loss: 18.8199 - val_q0.5_loss: 6.9911 - val_q0.8_loss: 7.6170 - val_q0.95_loss: 4.3600
- learning_rate: 1.2500e-04
Restoring model weights from the end of the best epoch: 49.
```

In []:

```
In [33]: # =====
# 8. EVALUATION
# =====

print("\n" + "="*50)
print("EVALUATION")
print("="*50)

preds = model.predict(val_ds)
pred50 = preds[0].reshape(-1)
pred80 = preds[1].reshape(-1)
pred95 = preds[2].reshape(-1)

=====
EVALUATION
=====
28/28  1s 14ms/step
```

In []:

```
In [34]: # --- Metrics ---
mae_50 = mean_absolute_error(y_val, pred50)
mae_80 = mean_absolute_error(y_val, pred80)
mae_95 = mean_absolute_error(y_val, pred95)

rmse_50 = np.sqrt(mean_squared_error(y_val, pred50))
rmse_80 = np.sqrt(mean_squared_error(y_val, pred80))
rmse_95 = np.sqrt(mean_squared_error(y_val, pred95))

r2_50 = r2_score(y_val, pred50)
r2_80 = r2_score(y_val, pred80)
r2_95 = r2_score(y_val, pred95)

coverage50 = np.mean(y_val <= pred50)
coverage80 = np.mean(y_val <= pred80)
coverage95 = np.mean(y_val <= pred95)

def pinball(y, y_pred, q):
    e = y - y_pred
```

```

    return np.mean(np.maximum(q*e, (q-1)*e))

pinball_50 = pinball(y_val, pred50, 0.5)
pinball_80 = pinball(y_val, pred80, 0.8)
pinball_95 = pinball(y_val, pred95, 0.95)

```

In []:

```

In [35]: # --- Results Table ---
metrics_df = pd.DataFrame({
    "Metric": ["MAE", "RMSE", "R²", "Coverage", "Pinball Loss"],
    "P50": [mae_50, rmse_50, r2_50, coverage50, pinball_50],
    "P80": [mae_80, rmse_80, r2_80, coverage80, pinball_80],
    "P95": [mae_95, rmse_95, r2_95, coverage95, pinball_95],
})

print("\n PERFORMANCE METRICS:")
print(metrics_df.to_string(index=False))

```

```

PERFORMANCE METRICS:
   Metric      P50      P80      P95
MAE 13.909944 19.359085 45.520992
RMSE 31.318613 33.657577 60.424443
R²    0.255460  0.140098 -1.771459
Coverage 0.557997 0.818740 0.942795
Pinball Loss 6.954972 7.559422 4.297522

```

In []:

```

In [36]: # --- Classification Accuracy ---
def classify_delay(m):
    if m < 0:
        return "EARLY"
    elif m <= 15:
        return "ON_TIME"
    else:
        return "DELAYED"

true_c = np.array([classify_delay(x) for x in y_val])
pred_c = np.array([classify_delay(x) for x in pred50])

```

```
accuracy = np.mean(true_c == pred_c)
print(f"\nClassification Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")
```

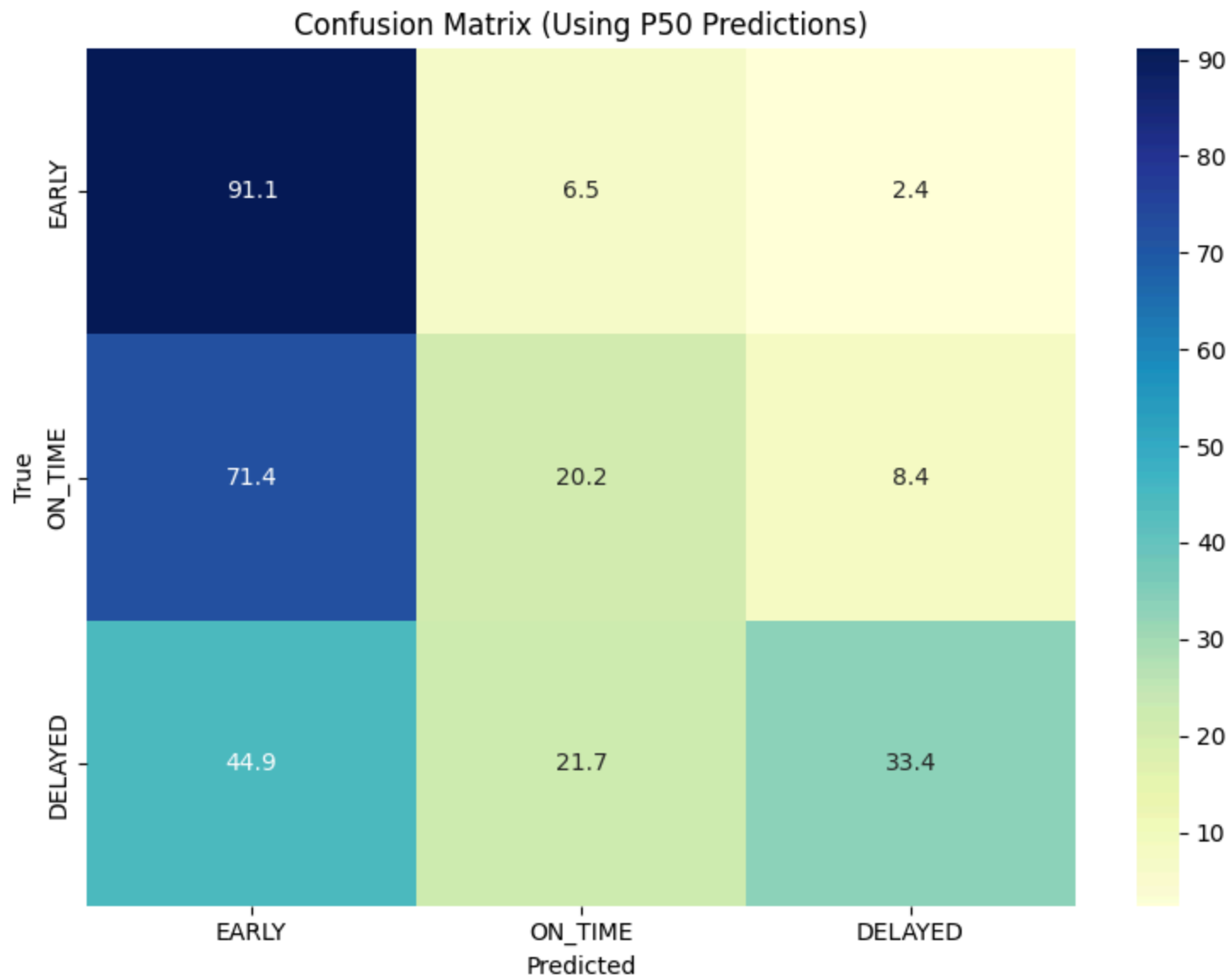
Classification Accuracy: 0.7174 (71.74%)

In []:

```
In [37]: # --- Confusion Matrix ---
from sklearn.metrics import confusion_matrix

labels = ["EARLY", "ON_TIME", "DELAYED"]
cm = confusion_matrix(true_c, pred_c, labels=labels)
cm_norm = cm.astype(float) / cm.sum(axis=1, keepdims=True) * 100

plt.figure(figsize=(8, 6))
sns.heatmap(cm_norm, annot=np.round(cm_norm, 1), fmt=".1f", cmap="YlGnBu",
            xticklabels=labels, yticklabels=labels)
plt.title("Confusion Matrix (Using P50 Predictions)")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.tight_layout()
plt.show()
```



In []:

In [38]:

```
# =====  
# 9. VISUALIZATIONS  
# =====
```

```
# --- Training history ---
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.plot(history.history['loss'], label='Train Loss')
ax1.plot(history.history['val_loss'], label='Val Loss')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Loss')
ax1.set_title('Training History')
ax1.legend()
ax1.grid(True, alpha=0.3)

# --- Distribution comparison ---
ax2.hist(y_val, bins=50, alpha=0.5, label='True', density=True)
ax2.hist(pred50, bins=50, alpha=0.5, label='Predicted (P50)', density=True)
ax2.set_xlabel('Delay (minutes)')
ax2.set_ylabel('Density')
ax2.set_title('Delay Distribution: True vs Predicted')
ax2.legend()
ax2.grid(True, alpha=0.3)

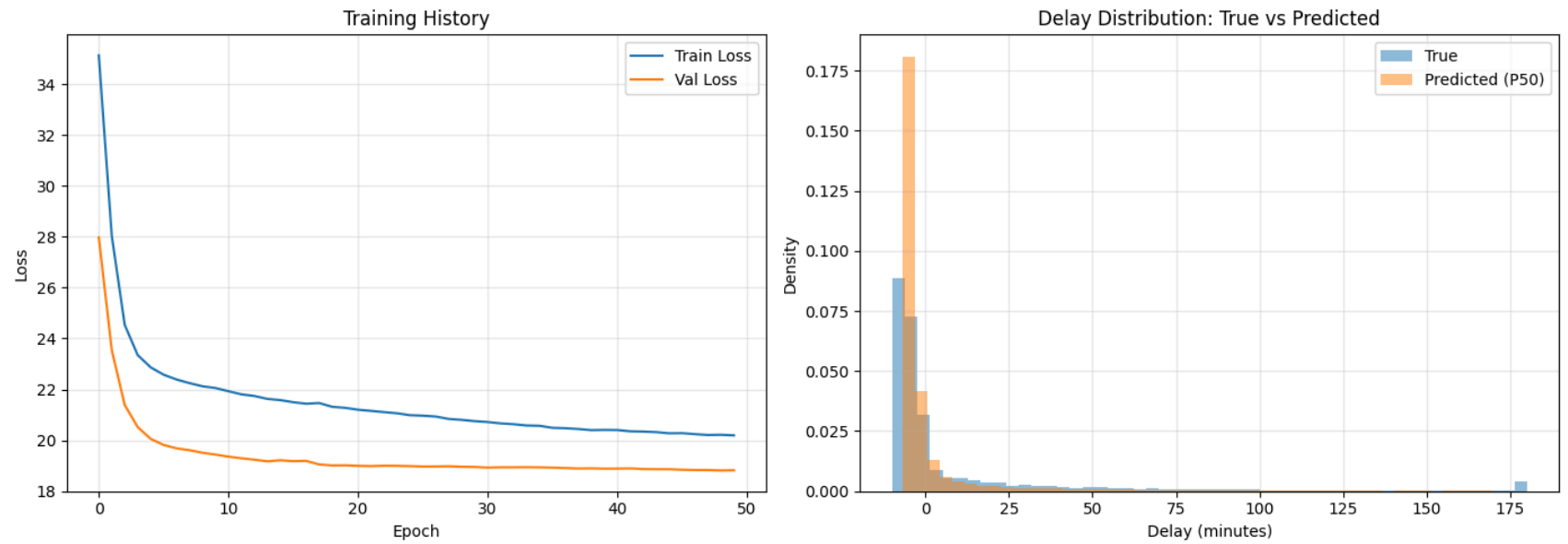
plt.tight_layout()
plt.show()

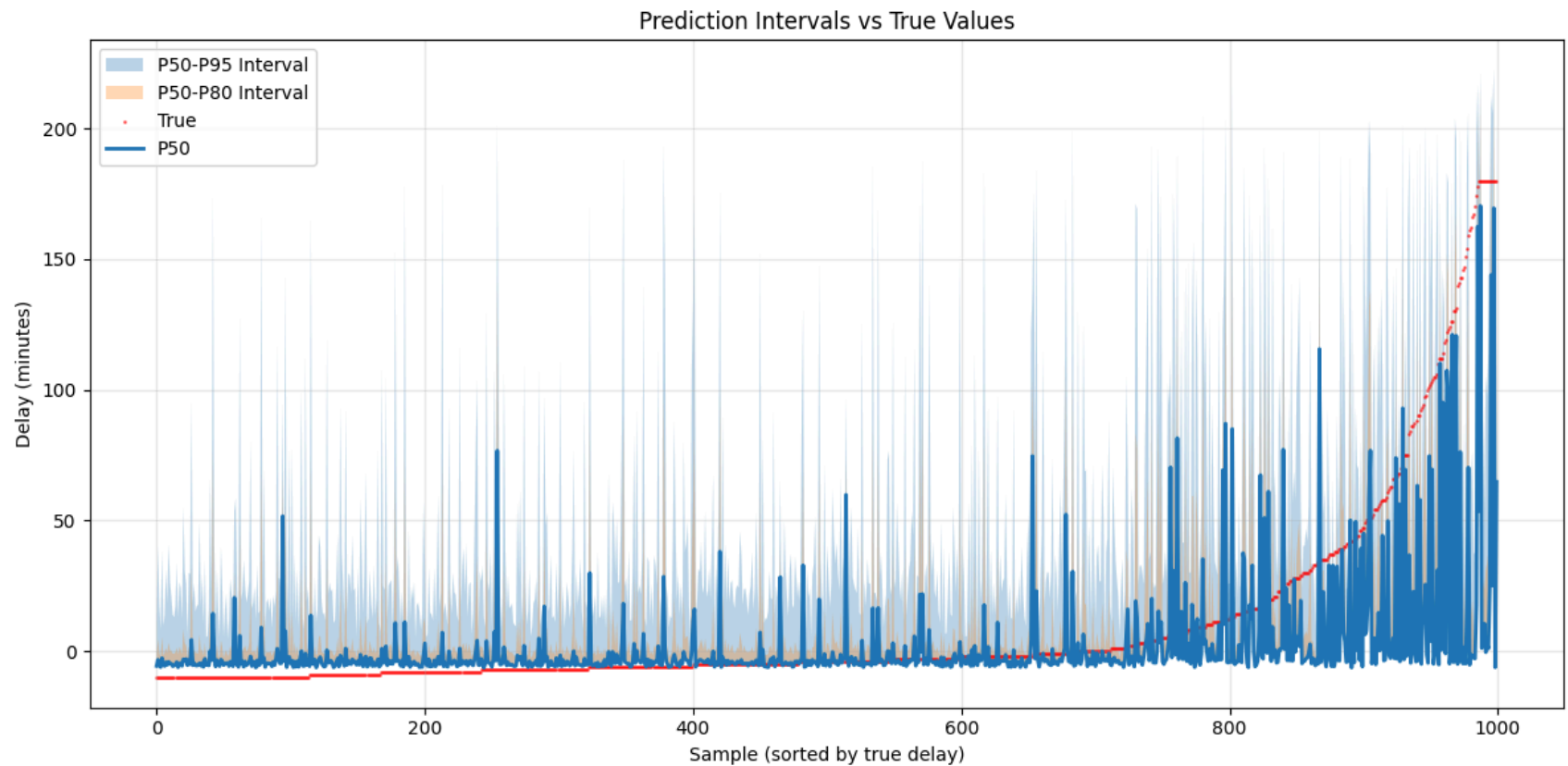
# --- Prediction intervals ---
plt.figure(figsize=(12, 6))
sample_size = min(1000, len(y_val))
idx = np.random.choice(len(y_val), sample_size, replace=False)
idx_sorted = idx[np.argsort(y_val[idx])]

plt.fill_between(range(sample_size), pred50[idx_sorted], pred95[idx_sorted],
                 alpha=0.3, label='P50-P95 Interval')
plt.fill_between(range(sample_size), pred50[idx_sorted], pred80[idx_sorted],
                 alpha=0.3, label='P50-P80 Interval')
plt.scatter(range(sample_size), y_val[idx_sorted], s=1, alpha=0.5, label='True', c='red')
plt.plot(pred50[idx_sorted], label='P50', linewidth=2)

plt.xlabel('Sample (sorted by true delay)')
plt.ylabel('Delay (minutes)')
plt.title('Prediction Intervals vs True Values')
plt.legend()
plt.grid(True, alpha=0.3)
```

```
plt.tight_layout()  
plt.show()
```





Make Number of features and bar graph of feature importance!

```
In [43]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import mean_absolute_error
import tensorflow as tf

print("="*60)
print("FEATURE IMPORTANCE ANALYSIS")
print("="*60)
```

```

# count features

all_features = numeric_features + cat_int_features + cat_str_features

print(f"\nTOTAL NUMBER OF FEATURES: {len(all_features)}")
print(f"    - Numeric features: {len(numeric_features)}")
print(f"    - Integer categorical: {len(cat_int_features)}")
print(f"    - String categorical: {len(cat_str_features)}")

print("\nFEATURE LIST:")
print("\nNumeric Features:")
for i, feat in enumerate(numeric_features, 1):
    print(f"    {i}. {feat}")

print("\nInteger Categorical Features:")
for i, feat in enumerate(cat_int_features, 1):
    print(f"    {i}. {feat}")

print("\nString Categorical Features:")
for i, feat in enumerate(cat_str_features, 1):
    print(f"    {i}. {feat}")

# feature categories

feature_categories = {
    'Temporal (Cyclical)': ['HOUR_SIN', 'HOUR_COS', 'MONTH_SIN', 'MONTH_COS', 'DAY_SIN', 'DAY_COS'],
    'Temporal (Original)': ['YEAR', 'DAY_OF_MONTH'],
    'Weather': ['WEATHER_FLAG', 'WEATHER_RUSH_HOUR'],
    'Congestion': ['ORIGIN_HOURLY_FLTS', 'DEST_HOURLY_FLTS', 'ORIGIN_DAILY_AVG_DEP_DELAY'],
    'Historical Performance': [
        'TAIL_DELAY_LAG1', 'TAIL_DELAY_LAG2', 'TAIL_DELAY_LAG3',
        'TAIL_AVG_DELAY_7D', 'AIRLINE_HOUR_AVG_DELAY',
        'AIRLINE_DAILY_AVG_DELAY', 'ROUTE_AVG_DELAY'
    ],
    'Time Patterns': ['IS_MORNING_RUSH', 'IS_EVENING_RUSH', 'IS_WEEKEND', 'IS_HOLIDAY_SEASON'],
    'Categorical': cat_int_features + cat_str_features
}

print("\n" + "="*60)
print("FEATURES BY CATEGORY")
print("="*60)

```

```

category_counts = {}
for category, features in feature_categories.items():
    count = len(features)
    category_counts[category] = count
    print(f"\n{category}: {count} features")
    for feat in features:
        print(f"    - {feat}")

# permutatioin importance

# Get baseline performance
baseline_preds = model.predict(val_ds, verbose=0)
baseline_pred50 = baseline_preds[0].reshape(-1)
baseline_mae = mean_absolute_error(y_val, baseline_pred50)

print(f"\n Baseline MAE: {baseline_mae:.3f} minutes")

# calculate importance for each feature
importances = {}

for feature in all_features:
    # Create a copy of validation data
    X_val_permuted = X_val.copy()

    # Shuffle this feature (break its relationship with target)
    X_val_permuted[feature] = np.random.permutation(X_val_permuted[feature].values)

    # Create dataset with permuted feature
    val_ds_permuted = df_to_ds(X_val_permuted, y_val, shuffle=False)

    # Get predictions
    preds_permuted = model.predict(val_ds_permuted, verbose=0)
    pred50_permuted = preds_permuted[0].reshape(-1)

    # Calculate MAE with permuted feature
    mae_permuted = mean_absolute_error(y_val, pred50_permuted)

    # Importance = increase in error when feature is shuffled
    importance = mae_permuted - baseline_mae
    importances[feature] = importance

```

```

    print(f"    {feature}: {importance:.4f}")

# sort and display top features

# convert to dataframe and sort
importance_df = pd.DataFrame({
    'Feature': list(importances.keys()),
    'Importance': list(importances.values())
}).sort_values('Importance', ascending=False)

print("\n" + "="*60)
print("TOP 20 MOST IMPORTANT FEATURES")
print("="*60)

top_20 = importance_df.head(20)
for idx, row in top_20.iterrows():
    print(f"{row['Feature']:40s} → +{row['Importance']:.4f} min MAE when removed")

# visualizations

# Set style
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (14, 10)

# top features
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# top bar chart
ax1 = axes[0, 0]
top_20_plot = importance_df.head(20).sort_values('Importance')
colors = ['#d62728' if x > 0.5 else '#ff7f0e' if x > 0.2 else '#2ca02c'
          for x in top_20_plot['Importance']]
top_20_plot.plot(x='Feature', y='Importance', kind='barh', ax=ax1, color=colors, legend=False)
ax1.set_xlabel('Importance (MAE increase when removed)', fontsize=12, fontweight='bold')
ax1.set_ylabel('Feature', fontsize=12, fontweight='bold')
ax1.set_title('Top 20 Most Important Features', fontsize=14, fontweight='bold', pad=20)
ax1.grid(True, alpha=0.3, axis='x')

# val labels
for i, v in enumerate(top_20_plot['Importance']):
    ax1.text(v + 0.01, i, f'{v:.3f}', va='center', fontsize=9)

```

```

# importance category
ax2 = axes[0, 1]
category_importance = {}
for category, features in feature_categories.items():
    cat_importance = sum([importances.get(f, 0) for f in features if f in importances])
    category_importance[category] = cat_importance

cat_imp_df = pd.DataFrame({
    'Category': list(category_importance.keys()),
    'Total Importance': list(category_importance.values())
}).sort_values('Total Importance', ascending=False)

cat_colors = plt.cm.Set3(range(len(cat_imp_df)))
cat_imp_df.plot(x='Category', y='Total Importance', kind='barh', ax=ax2, color=cat_colors, legend=False)
ax2.set_xlabel('Total Importance', fontsize=12, fontweight='bold')
ax2.set_ylabel('Category', fontsize=12, fontweight='bold')
ax2.set_title('Feature Importance by Category', fontsize=14, fontweight='bold', pad=20)
ax2.grid(True, alpha=0.3, axis='x')

# Add value labels
for i, v in enumerate(cat_imp_df['Total Importance']):
    ax2.text(v + 0.01, i, f'{v:.3f}', va='center', fontsize=9)

# feature count category
ax3 = axes[1, 0]
cat_count_df = pd.DataFrame({
    'Category': list(category_counts.keys()),
    'Count': list(category_counts.values())
}).sort_values('Count', ascending=False)

cat_count_df.plot(x='Category', y='Count', kind='barh', ax=ax3, color='skyblue', legend=False)
ax3.set_xlabel('Number of Features', fontsize=12, fontweight='bold')
ax3.set_ylabel('Category', fontsize=12, fontweight='bold')
ax3.set_title('Feature Count by Category', fontsize=14, fontweight='bold', pad=20)
ax3.grid(True, alpha=0.3, axis='x')

# val labels
for i, v in enumerate(cat_count_df['Count']):
    ax3.text(v + 0.2, i, f'{int(v)}', va='center', fontsize=10, fontweight='bold')

# importance distribution

```

```

ax4 = axes[1, 1]
importance_values = importance_df['Importance'].values
ax4.hist(importance_values, bins=30, color='steelblue', alpha=0.7, edgecolor='black')
ax4.axvline(importance_values.mean(), color='red', linestyle='--', linewidth=2,
            label=f'Mean: {importance_values.mean():.3f}')
ax4.axvline(np.median(importance_values), color='orange', linestyle='--', linewidth=2,
            label=f'Median: {np.median(importance_values):.3f}')
ax4.set_xlabel('Importance Value', fontsize=12, fontweight='bold')
ax4.set_ylabel('Frequency', fontsize=12, fontweight='bold')
ax4.set_title('Distribution of Feature Importance', fontsize=14, fontweight='bold', pad=20)
ax4.legend(fontsize=10)
ax4.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# stats summary

print("\n" + "="*60)
print("FEATURE IMPORTANCE SUMMARY")
print("="*60)

print(f"\nBaseline Model MAE: {baseline_mae:.3f} minutes")
print(f"\nMost Important Feature: {importance_df.iloc[0]['Feature']}")
print(f"    → Removing it increases MAE by {importance_df.iloc[0]['Importance']:.3f} minutes")
print(f"\nLeast Important Feature: {importance_df.iloc[-1]['Feature']}")
print(f"    → Removing it changes MAE by {importance_df.iloc[-1]['Importance']:.3f} minutes")

print(f"\nAverage Importance: {importance_values.mean():.3f} minutes")
print(f"Median Importance: {np.median(importance_values):.3f} minutes")
print(f"Max Importance: {importance_values.max():.3f} minutes")
print(f"Min Importance: {importance_values.min():.3f} minutes")

# Identify critical features (importance > 0.5)
critical_features = importance_df[importance_df['Importance'] > 0.5]
print(f"\n🔴 Critical Features (importance > 0.5 min): {len(critical_features)}")
for _, row in critical_features.iterrows():
    print(f"    - {row['Feature']:40s} → +{row['Importance']:.3f} min")

# Identify moderate features (0.2 < importance < 0.5)

```

```
moderate_features = importance_df[(importance_df['Importance'] > 0.2) &
                                   (importance_df['Importance'] <= 0.5)]
print(f"\n🟡 Moderate Features (0.2-0.5 min): {len(moderate_features)}")

# Identify minor features (importance < 0.2)
minor_features = importance_df[importance_df['Importance'] <= 0.2]
print(f"\n🟢 Minor Features (< 0.2 min): {len(minor_features)}")

print("\n" + "="*60)
```

```
=====
FEATURE IMPORTANCE ANALYSIS
=====
```

TOTAL NUMBER OF FEATURES: 27

- Numeric features: 24
- Integer categorical: 2
- String categorical: 1

FEATURE LIST:

Numeric Features:

1. HOUR_SIN
2. HOUR_COS
3. MONTH_SIN
4. MONTH_COS
5. DAY_SIN
6. DAY_COS
7. YEAR
8. DAY_OF_MONTH
9. WEATHER_FLAG
10. WEATHER_RUSH_HOUR
11. ORIGIN_HOURLY_FLTS
12. DEST_HOURLY_FLTS
13. ORIGIN_DAILY_AVG_DEP_DELAY
14. TAIL_DELAY_LAG1
15. TAIL_DELAY_LAG2
16. TAIL_DELAY_LAG3
17. TAIL_AVG_DELAY_7D
18. AIRLINE_HOUR_AVG_DELAY
19. AIRLINE_DAILY_AVG_DELAY
20. ROUTE_AVG_DELAY
21. IS_MORNING_RUSH
22. IS_EVENING_RUSH
23. IS_WEEKEND
24. IS_HOLIDAY_SEASON

Integer Categorical Features:

1. OP_CARRIER_AIRLINE_ID
2. DEST_AIRPORT_ID

String Categorical Features:

1. TAIL_NUM

```
=====
FEATURES BY CATEGORY
=====
```

Temporal (Cyclical): 6 features

- HOUR_SIN
- HOUR_COS
- MONTH_SIN
- MONTH_COS
- DAY_SIN
- DAY_COS

Temporal (Original): 2 features

- YEAR
- DAY_OF_MONTH

Weather: 2 features

- WEATHER_FLAG
- WEATHER_RUSH_HOUR

Congestion: 3 features

- ORIGIN_HOURLY_FLTS
- DEST_HOURLY_FLTS
- ORIGIN_DAILY_AVG_DEP_DELAY

Historical Performance: 7 features

- TAIL_DELAY_LAG1
- TAIL_DELAY_LAG2
- TAIL_DELAY_LAG3
- TAIL_AVG_DELAY_7D
- AIRLINE_HOUR_AVG_DELAY
- AIRLINE_DAILY_AVG_DELAY
- ROUTE_AVG_DELAY

Time Patterns: 4 features

- IS_MORNING_RUSH
- IS_EVENING_RUSH
- IS_WEEKEND
- IS_HOLIDAY_SEASON

Categorical: 3 features

- OP_CARRIER_AIRLINE_ID
- DEST_AIRPORT_ID
- TAIL_NUM

Baseline MAE: 13.910 minutes

HOUR_SIN: 0.6911
HOUR_COS: 0.2283
MONTH_SIN: 0.0395
MONTH_COS: 0.0357
DAY_SIN: 0.0108
DAY_COS: 0.0126
YEAR: 0.0223
DAY_OF_MONTH: 0.0041
WEATHER_FLAG: 0.1125
WEATHER_RUSH_HOUR: 0.0243
ORIGIN_HOURLY_FLTS: 0.5873
DEST_HOURLY_FLTS: 0.1063
ORIGIN_DAILY_AVG_DEP_DELAY: 0.1844
TAIL_DELAY_LAG1: 2.1932
TAIL_DELAY_LAG2: 0.0332
TAIL_DELAY_LAG3: -0.0038
TAIL_AVG_DELAY_7D: 0.0034
AIRLINE_HOUR_AVG_DELAY: 0.1577
AIRLINE_DAILY_AVG_DELAY: 2.5081
ROUTE_AVG_DELAY: 0.0157
IS_MORNING_RUSH: 0.1221
IS_EVENING_RUSH: 0.0367
IS_WEEKEND: 0.0180
IS_HOLIDAY_SEASON: 0.0055
OP_CARRIER_AIRLINE_ID: -0.0166
DEST_AIRPORT_ID: 0.4307
TAIL_NUM: -0.0004

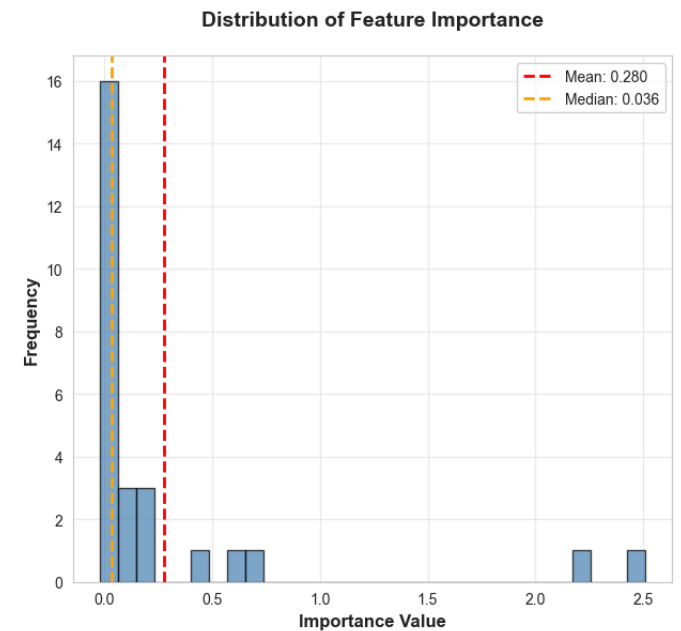
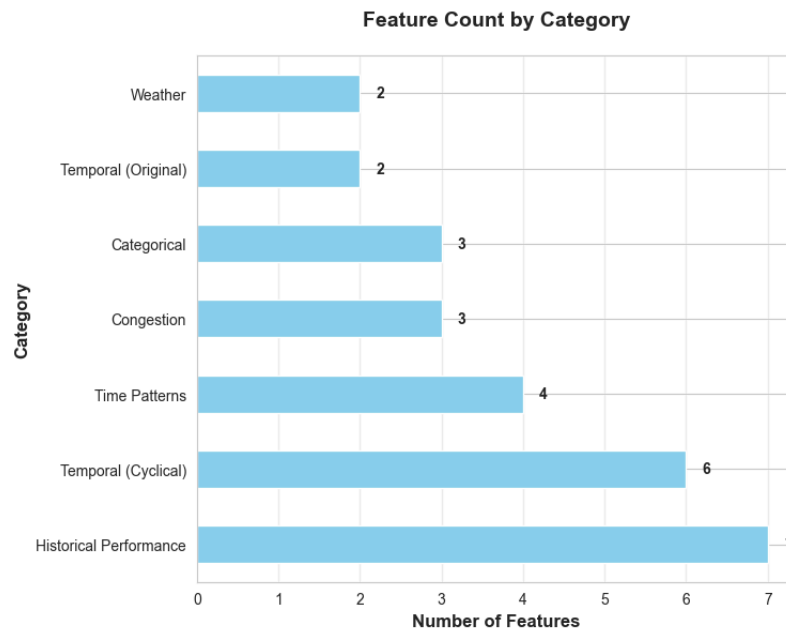
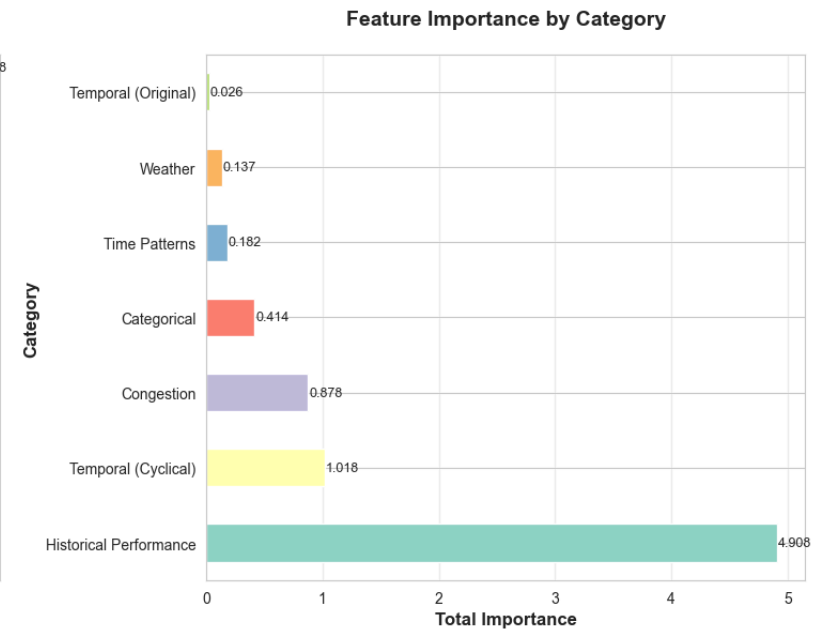
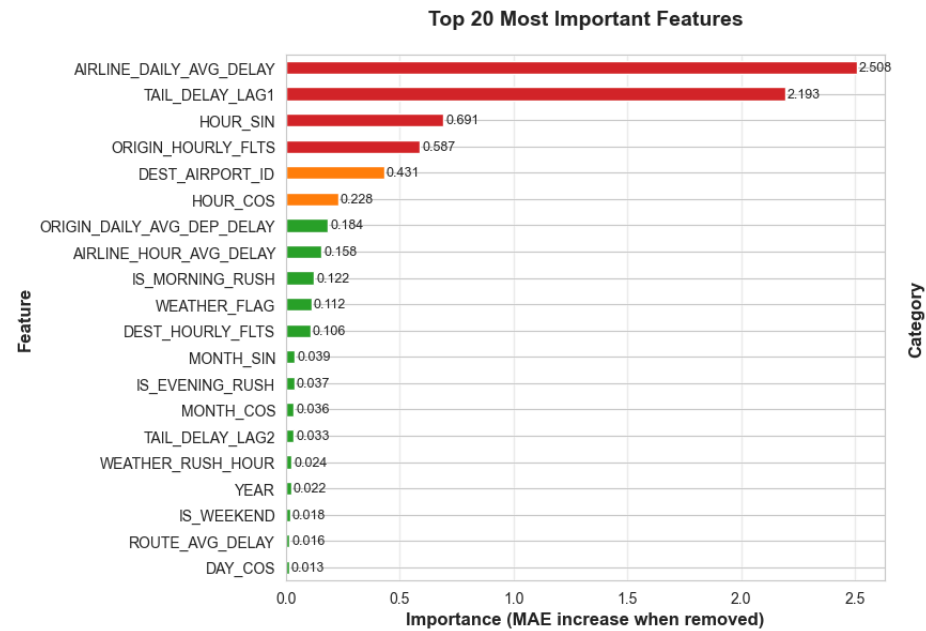
=====

TOP 20 MOST IMPORTANT FEATURES

=====

AIRLINE_DAILY_AVG_DELAY	→ +2.5081 min MAE when removed
TAIL_DELAY_LAG1	→ +2.1932 min MAE when removed
HOUR_SIN	→ +0.6911 min MAE when removed
ORIGIN_HOURLY_FLTS	→ +0.5873 min MAE when removed
DEST_AIRPORT_ID	→ +0.4307 min MAE when removed

HOUR_COS	→ +0.2283 min MAE when removed
ORIGIN_DAILY_AVG_DEP_DELAY	→ +0.1844 min MAE when removed
AIRLINE_HOUR_AVG_DELAY	→ +0.1577 min MAE when removed
IS_MORNING_RUSH	→ +0.1221 min MAE when removed
WEATHER_FLAG	→ +0.1125 min MAE when removed
DEST_HOURLY_FLTS	→ +0.1063 min MAE when removed
MONTH_SIN	→ +0.0395 min MAE when removed
IS_EVENING_RUSH	→ +0.0367 min MAE when removed
MONTH_COS	→ +0.0357 min MAE when removed
TAIL_DELAY_LAG2	→ +0.0332 min MAE when removed
WEATHER_RUSH_HOUR	→ +0.0243 min MAE when removed
YEAR	→ +0.0223 min MAE when removed
IS_WEEKEND	→ +0.0180 min MAE when removed
ROUTE_AVG_DELAY	→ +0.0157 min MAE when removed
DAY_COS	→ +0.0126 min MAE when removed



```
=====
FEATURE IMPORTANCE SUMMARY
=====
```

Baseline Model MAE: 13.910 minutes

Most Important Feature: AIRLINE_DAILY_AVG_DELAY
→ Removing it increases MAE by 2.508 minutes

Least Important Feature: OP_CARRIER_AIRLINE_ID
→ Removing it changes MAE by -0.017 minutes

Average Importance: 0.280 minutes

Median Importance: 0.036 minutes

Max Importance: 2.508 minutes

Min Importance: -0.017 minutes

● Critical Features (importance > 0.5 min): 4

- AIRLINE_DAILY_AVG_DELAY	→ +2.508 min
- TAIL_DELAY_LAG1	→ +2.193 min
- HOUR_SIN	→ +0.691 min
- ORIGIN_HOURLY_FLTS	→ +0.587 min

● Moderate Features (0.2-0.5 min): 2

● Minor Features (< 0.2 min): 21

```
=====
```

In []: